Scale-Invariant State Representation and Maximum-Entropy Deep Reinforcement Learning for Task Offloading in Vehicular Edge Computing

Antônio Sérgio de Sousa Vieira[1,2*], Alisson Barbosa de Souza[1,3] and Joaquim Celestino Júnior[1]

[1*]Intelligent Networks and Systems Group (INETSYS), State University of Ceará, Fortaleza, Brazil.
[2]Federal Institute of Education, Science and Technology, Itapipoca, Brazil.
[3]Federal University of Ceará, Quixadá, Brazil.

*Corresponding author(s). E-mail(s): sergio.vieira@ifce.edu.br;
Contributing authors: alisson@ufc.br; joaquim.celestino@uece.br;

Abstract

Task offloading in Vehicular Edge Computing (VEC) networks is a critical mechanism to satisfy the stringent latency and energy constraints of advanced autonomous applications. The high mobility of vehicles creates an environment in which the number of available edge servers and neighboring vehicles changes continuously, so the observation space is *dimensionally variable*. This variability prevents standard Deep Reinforcement Learning (DRL) agents from generalizing across vehicular densities without costly retraining. To address this challenge, this paper proposes **SAC-TOPSIS**, a hybrid decision-making architecture that integrates the Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS) with the Soft Actor-Critic (SAC) algorithm. The TOPSIS module evaluates every candidate offloading node—each characterized by Job Completion Time (JCT), energy consumption, and physical distance—using $\boldsymbol{L_1}$-Manhattan distances and weights $\mathbf{[0.4, 0.3, 0.3]}$, and then compresses the variable-size observation into a *fixed* 6-dimensional, scale-invariant state vector. A SAC agent receives this compact representation and learns a binary policy: execute locally or offload to the top-ranked candidate. Entropy regularization prevents the policy from collapsing into deterministic local-only or offload-only modes. Extensive simulation across 601 heterogeneous Vehicle-to-Everything (V2X) and 5G scenarios demonstrates that the TOPSIS integration consistently improves every tested DRL family: SAC improves from **4.47%** to **44.36%** (**+39.9** pp), TD3 from **41.62%** to **43.01%** (**+1.4** pp), and PPO from **41.61%** to **41.95%** (**+0.3** pp) in deadline compliance rate. Among all evaluated policies, SAC-TOPSIS achieves the highest compliance (**44.36%**) and the highest Success-Weighted Quality (SWQ) index (**0.1450**). Crucially, the model is trained on a single density regime and generalizes *zero-shot* to unseen vehicular densities ranging from 10 to 30 tasks/s.

Keywords: Task offloading, Vehicular Edge Computing, Soft Actor-Critic, TOPSIS, Multi-criteria state compression, Zero-shot generalization

1 Introduction

Context and motivation.

The steady cost reduction of semiconductor and communication technologies has enabled a new generation of intelligent vehicle applications demanding simultaneously high efficiency, usefulness, reliability, and safety [1]. Advanced driver assistance systems, real-time object detection, cooperative perception, and V2X path planning impose latency budgets often below 10 ms [2], requiring enormous on-demand computation at the network edge. Fifth-generation New Radio (5G NR) and its successor 6G provide unprecedented communication throughput and ultra-low radio latency, effectively eliminating the transmission bottleneck for short payloads. However, lower propagation latency does not eliminate the *processing* challenge: modern vehicular applications generate massive volumes of sensor data—LiDAR point clouds, HD maps, multi-camera streams—whose end-to-end response time and energy cost remain critical constraints [3, 4].

MEC and VEC as solutions.

Mobile Edge Computing (MEC) and its vehicular specialization, Vehicular Edge Computing (VEC), address these constraints by co-locating compute resources with the radio access network [5]. Vehicles can offload computation-intensive tasks to Road-Side Unit (RSU) servers via 5G C-V2X or to neighboring vehicles via direct V2V sidelink, dramatically reducing both response time and onboard energy consumption [**?** 6].

Heterogeneous OBU capacity and cooperative computing.

A key observation in modern vehicular environments is that On-Board Units (OBUs) are *heterogeneous*: high-end vehicles equipped with powerful processing units (GPUs, NPUs) may temporarily carry underutilized capacity, while compact vehicles operate near their computational limits. This heterogeneity creates cooperative computing opportunities in which resource-rich neighbors can absorb tasks from resource-constrained ones, improving Quality of Service (QoS) for the entire Intelligent Transportation System (ITS) [7]. Realizing this potential, however, requires offloading policies that are aware of the network's shifting topology and can reason about multi-hop, multi-criteria trade-offs in real time.

Challenges and research gaps.

Despite a growing body of literature, several critical gaps remain open, as systematically identified by Miriyala and Chirra [8]. Most evaluation studies rely on simplified, static scenarios—fixed number of vehicles, uniform task distributions, idealized channel models—that fail to capture the high dynamism of real vehicular networks. Consequently, policies that perform well in synthetic benchmarks often degrade sharply when vehicular density or traffic regime changes.

Why classical strategies fall short.

Task offloading is fundamentally a multi-criteria optimization problem. *Heuristics* such as greedy strategies have been widely deployed due to their simplicity [9], but they evaluate only the current network snapshot and are unable to anticipate queue saturation or future task arrivals. *Metaheuristics* (genetic algorithms, particle swarm optimization, ant colony optimization) offer stronger global search guarantees and have been applied to static offloading [10, 11, 12], yet they are fundamentally ill-suited to vehicular environments: their population-based search requires tens to hundreds of fitness evaluations per decision instant, which is incompatible with sub-second deadlines; they must be rerun from scratch every time the topology changes; and their initialization becomes increasingly non-trivial as the network grows.

Deep Reinforcement Learning and its limitations.

Deep Reinforcement Learning (DRL) is a natural fit for task offloading because it learns a *policy*—a mapping from observations to actions—that implicitly accounts for future consequences, not just the current state [**?** 13]. Unlike heuristics and metaheuristics, a DRL agent can exploit temporal correlations in traffic patterns and queue dynamics learned from experience. Nevertheless, standard DRL architectures share a critical weakness: they require *retraining* whenever the operational scenario changes [8]. Because the neural network input is a fixed-size vector,

any change in the number of candidate offloading nodes—caused by vehicles entering or leaving communication range—invalidates the trained model. Zero-padding absent candidates is a common workaround but introduces spurious inputs that destabilize learning and reduce the policy's ability to generalize.

Our prior work and the path to SAC-TOPSIS.

In our previous work, we proposed FOFF [14], a Fuzzy-TOPSIS framework that ranks edge servers in a *static* VEC scenario using linguistic criteria weights to handle uncertainty. FOFF demonstrated that multi-criteria methods can achieve strong energy efficiency and QoS without any training phase. Its limitation is precisely the lack of adaptation: Fuzzy-TOPSIS selects the best candidate by a fixed scoring rule and cannot improve from experience or adjust to non-stationary traffic. Building on these insights, we now propose **SAC-TOPSIS**: a hybrid architecture that delegates the multi-criteria *candidate evaluation* to TOPSIS while delegating the long-horizon *policy optimization* to the Soft Actor-Critic (SAC) algorithm, combining the local optimality of MCDM with the forward-looking capability of DRL.

Contributions.

The main contributions of this paper are:

1. **Challenging, realistic evaluation scenario.** We evaluate all policies on 601 heterogeneous VEC scenarios spanning five vehicular density regimes (10–30 tasks/s), with realistic 5G and V2V channel models, heterogeneous OBU frequencies, and per-task deadlines drawn from a continuous distribution.
2. **Scale-invariant state representation.** TOPSIS compresses the variable-size candidate set into a fixed 6D vector whose components are normalized ratios and relative differences, making it valid regardless of the number of available candidates. This representation enables *zero-shot generalization*: a model trained on one density regime transfers directly to unseen regimes without retraining.
3. **Low-complexity decision architecture.** By reducing the action space to a single bit (local vs. rank-1 offload), the agent avoids exponential action-space growth and runs inference in $O(1)$ neural-network forward passes per task—suitable for onboard real-time deployment.
4. **Principled reward shaping.** The deadline-aware reward function provides a continuous gradient signal in both the success and failure regimes, preventing reward sparsity and teaching the agent to minimize tardiness rather than merely avoid it.
5. **Complementary decision horizons.** TOPSIS provides the best *local* solution given current network conditions; SAC learns the best *long-term* policy by looking ahead through accumulated experience. The combination yields results that neither component can achieve alone.

Evaluated across 601 scenarios, SAC-TOPSIS achieves a deadline compliance rate of 44.36% and SWQ of 0.1450—the best among all DRL and heuristic baselines—while the raw-state SAC-Base achieves only 4.47% under the same training budget.

The remainder of the paper is structured as follows. Section 2 reviews related work. Section 3 formalizes the system model and optimization objective. Section 4 describes the SAC-TOPSIS architecture in detail. Section 5 presents the experimental evaluation. Section 6 concludes the paper.

2 Related Work

2.1 Task Offloading in VEC

Task offloading optimization has been extensively studied as a mixed latency- energy trade-off [15, 6]. Early works relied on deterministic multi-criteria decision-making (MCDM) methods. The FOFF framework [14] applied Fuzzy-TOPSIS to rank edge servers by latency and energy without any training phase, achieving significant energy savings. However, purely rule-based methods cannot adapt to non-stationary queuing dynamics or learn from historical performance. Hybrid approaches that combine MCDM pre-filtering with a learned policy are an active research direction [1].

2.2 Deep Reinforcement Learning for VEC

DRL-based offloading has gained traction for its ability to adapt online. Deep Q-Networks (DQN) and their dueling variants address the discrete-action case [16], while actor-critic algorithms (PPO, DDPG, TD3) handle continuous or high-dimensional settings [17]. A common limitation is the fixed-input-size assumption: most models use zero-padding to accommodate up to a maximum number of candidates, introducing spurious symmetry-breaking artifacts and reducing generalization. Off-policy algorithms (SAC, TD3) offer better sample efficiency than on-policy methods, but are still susceptible to representation collapse when the state is uninformative [18].

2.3 Positioning of SAC-TOPSIS

Prior work has combined MCDM methods with DRL for MEC, but these typically treat MCDM as a post-processing filter (ranking the DRL action set) rather than as a *state-engineering* step. SAC-TOPSIS is, to our knowledge, the first architecture to use TOPSIS *inside the observation pipeline*: its output is not a decision but a compact, scale-invariant state encoding that enables DRL training without zero-padding or fixed topology assumptions. Crucially, this state compression is *algorithm-agnostic*: our experiments show that pairing the TOPSIS state with SAC, TD3, and PPO each yields consistent improvements over their respective raw-state baselines, confirming that the representation itself—rather than any particular DRL algorithm—is the primary driver of generalization.

3 System Model and Problem Formulation

3.1 Network Model

We consider a heterogeneous VEC environment with one client vehicle v_0, one RSU server accessible via 5G C-V2X (uplink bandwidth B_{V2I}), and a time-varying set $\mathcal{V}(t)$ of neighboring vehicles accessible via V2V sidelink (bandwidth B_{V2V}). The RSU has a service queue of capacity Q_{RSU}; each vehicle $v \in \mathcal{V}(t)$ has a local queue Q_v.

3.2 Task Model

Each task $\tau_j = (C_j, D_j, s_j)$ is characterized by its CPU complexity C_j (cycles), deadline D_j (seconds), and input data size s_j (bits). Three execution *modes* are available:

- **Local ($m = 0$):** executed on v_0 with frequency f_{loc}.
- **RSU ($m = 1$):** uploaded to the RSU and executed at f_{RSU}.
- **V2V ($m = 2$):** offloaded to a neighboring vehicle $v^* \in \mathcal{V}(t)$.

A mode m is *feasible* if the corresponding peer is within communication range and its queue is not full.

3.3 Latency Model

The Job Completion Time of mode m for task τ_j is

$$\mathrm{JCT}_j^{(m)} = t_j^{\mathrm{tx},(m)} + t_j^{\mathrm{queue},(m)} + \frac{C_j}{f^{(m)}}, \quad (1)$$

where $t_j^{\mathrm{tx},(m)}$ is the transmission time (upload), and $t_j^{\mathrm{queue},(m)}$ is the residual waiting time in the target queue, estimated as the remaining execution time of tasks already in the queue.

3.4 Energy Model

The energy consumed for mode m is

$$E_j^{(m)} = \alpha_c^{(m)} \left[V^{(m)}\right]^2 C_j + P_{\mathrm{TX}}^{(m)}\, t_j^{\mathrm{tx},(m)}, \quad (2)$$

where $\alpha_c^{(m)}$ is the chip capacitance, $V^{(m)}$ is the supply voltage at frequency $f^{(m)}$, and $P_{\mathrm{TX}}^{(m)}$ is the transmit power. For the local mode, the transmit term is zero.

3.5 Optimization Objective

The per-task QoS contribution is

$$\Phi_j = \alpha\, \mathrm{JCT}_j + \beta\, E_j + \gamma\, \mathbf{1}[\mathrm{JCT}_j > D_j]\, \mathrm{JCT}_j, \quad (3)$$

with $\alpha = \beta = 0.5$ and a deadline-miss penalty coefficient $\gamma = 0.015$. The system objective is to minimize $\sum_j \Phi_j$ while maximizing the *deadline compliance rate* $\hat{\tau} = |\{j : \mathrm{JCT}_j \le D_j\}|/N$.

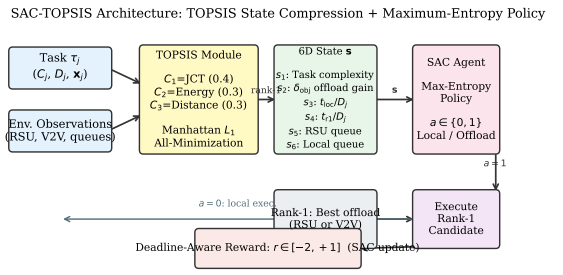

Fig. 1 SAC-TOPSIS architecture. TOPSIS compresses the variable-size candidate set into a fixed 6D scale-invariant state vector. SAC learns a binary policy (local vs. rank-1 offload) on this representation with maximum-entropy regularization.

4 The SAC-TOPSIS Architecture

Figure 1 depicts the complete architecture. At each decision instant, the TOPSIS module processes the multi-criteria information about all feasible candidates and produces the 6D state vector that is passed to the SAC agent.

4.1 TOPSIS-Based State Compression

Decision matrix.

For task τ_j, we build a 3×3 decision matrix $\mathbf{X} \in \mathbb{R}^{3\times 3}_{\geq 0}$ whose rows correspond to the three candidate modes (local, RSU, V2V) and whose columns are the three *cost* criteria:

$$\mathbf{X} = \begin{bmatrix} \mathrm{JCT}_j^{(0)} & E_j^{(0)} & 0 \\ \mathrm{JCT}_j^{(1)} & E_j^{(1)} & d_{\mathrm{RSU}} \\ \mathrm{JCT}_j^{(2)} & E_j^{(2)} & d_{\mathrm{V2V}} \end{bmatrix}, \tag{4}$$

where d_{RSU} and d_{V2V} are the Euclidean distances to the RSU and the selected V2V peer, respectively. If a mode is infeasible its row is set to 10^6 to exclude it from the ideal solutions.

Weighted normalization.

Each column j is normalized by its Euclidean norm $\|\mathbf{X}_{:j}\|_2$, then multiplied by weight $w_j \in \{0.4, 0.3, 0.3\}$ (JCT, energy, distance), yielding the weighted normalized matrix $\widetilde{\mathbf{X}}$.

L_1-Manhattan TOPSIS.

We depart from classic TOPSIS by measuring distances to the positive ideal solution (PIS) A^+ and negative ideal solution (NIS) A^- using the *Manhattan* (L_1) norm:

$$D_i^+ = \textstyle\sum_k |\widetilde{x}_{ik} - A_k^+|, \qquad D_i^- = \textstyle\sum_k |\widetilde{x}_{ik} - A_k^-|. \tag{5}$$

The proximity score is then $\mathcal{C}_i = D_i^- / (D_i^+ + D_i^-)$. Because all criteria are cost-minimization objectives, A^+ is the column-wise minimum and A^- is the column-wise maximum of $\widetilde{\mathbf{X}}$. The L_1 metric is less sensitive to outliers—a critical property when queue estimates are noisy—and computationally lighter than L_2.

Rank-1 selection.

The rank-1 offload candidate is the feasible *non-local* mode with the highest $\mathcal{C}_i$:

$$m^* = \operatorname*{argmax}_{i \in \{1,2\}} \mathcal{C}_i \text{ s.t. mode } i \text{ is feasible.} \tag{6}$$

Ties are broken in favour of V2V to encourage load balancing. If no offload mode is feasible, m^* defaults to local execution regardless of the agent's action.

6D state vector.

The state $\mathbf{s} \in \mathbb{R}^6$ is assembled as:

$$\begin{aligned}
s_1 &= C_j / 10^{10} && \text{(task complexity, normalized)} \\
s_2 &= \mathrm{clip}\left(1 - \frac{\mathrm{cost}_{m^*}}{\mathrm{cost}_{\mathrm{loc}}}, -1, 1\right) && \text{(objective-level offload advantage} \\
s_3 &= \min\left(1,\ \mathrm{JCT}^{(0)}/D_j\right) && \text{(local urgency ratio)} \\
s_4 &= \min\left(1,\ \mathrm{JCT}^{(m^*)}/D_j\right) && \text{(rank-1 urgency ratio)} \\
s_5 &= \min(1,\ |Q_{\mathrm{RSU}}|/25) && \text{(RSU queue occupancy)} \\
s_6 &= \min(1,\ |Q_{v_0}|/25) && \text{(local vehicle queue occupancy),}
\end{aligned} \tag{7}$$

where $\mathrm{cost}_m = \alpha\,\mathrm{JCT}^{(m)} + \beta\,E^{(m)}$. All components lie in $[0, 1]$ or $[-1, 1]$, making $\mathbf{s}$ *scale-invariant*: it does not depend on the number of available candidates, only on the relative quality of the best one.

4.2 Maximum-Entropy Soft Actor-Critic

The SAC agent optimizes the maximum-entropy objective [18]:

$$\pi^* = \underset{\pi}{\operatorname{argmax}}\ \mathbb{E}_{\tau\sim\pi}\left[\sum_t \gamma^t \big(r_t + \alpha_T\, \mathcal{H}[\pi(\cdot \mid s_t)]\big)\right], \tag{8}$$

where α_T is the automatically tuned temperature parameter. The actor and critic networks are MLPs with two hidden layers of 256 units and ReLU activations. The actor outputs logits for the Bernoulli action distribution; the temperature α_T is adapted to a target entropy of $\log 2/2$ nats. The replay buffer holds 10^5 transitions; mini-batch size is 256; the learning rate is 3×10^{-4} for all networks.

Why binary action?

Delegating the *selection* of the offload destination to TOPSIS reduces the action space from $|\mathcal{V}(t)|+2$ dimensions to a single bit. This simplification is lossless from the agent's perspective because the relative ordering of candidates is already encoded in s_2–s_4.

4.3 Deadline-Aware Reward Function

The reward r_j for task τ_j is designed to guide the agent toward deadline compliance while providing a continuous gradient signal:

$$r_j = \begin{cases} 1.0 - 0.5 \cdot \rho_j & \text{if } \mathrm{JCT}_j \le D_j, \\ -1.0 - \min(1,\, \delta_j) & \text{otherwise,} \end{cases} \tag{9}$$

where

$$\rho_j = \tfrac{1}{2}\left(\frac{\mathrm{JCT}_j}{D_j} + \frac{E_j}{E_{\max,j}}\right) \in [0,1] \tag{10}$$

is a normalized cost ratio, and $\delta_j = (\mathrm{JCT}_j - D_j)/D_j$ is the relative deadline overrun. Successful executions yield $r_j \in [0.5, 1.0]$: faster, more efficient tasks receive higher rewards. Failed executions yield $r_j \in [-2, -1]$: the penalty grows with the degree of tardiness, providing a gradient that teaches the agent to *fail by the least amount possible*.

5 Performance Evaluation

5.1 Simulation Setup

All algorithms run on a custom discrete-event VEC simulator written in C++20 with `simcpp20` and the Armadillo linear-algebra library. Neural networks are trained and exported using LibTorch (PyTorch C++ API). Mobility scenarios are generated with realistic V2X parameters: vehicular speeds of $30\,\mathrm{km\,h^{-1}}$ to $120\,\mathrm{km\,h^{-1}}$, 5G uplink bandwidth 20 MHz, V2V sidelink 10 MHz, RSU frequency $f_{\mathrm{RSU}} = 2\,\mathrm{GHz}$, and onboard $f_{\mathrm{loc}} \in [0.5, 1.0]\,\mathrm{GHz}$. Task deadlines follow a truncated Gaussian over $[0.05, 2.0]$ s; CPU complexity $C_j \sim \mathrm{Uniform}[10^8, 10^{10}]$ cycles.

Training.

All DRL agents are trained for 400 episodes on scenarios with a fixed vehicular density of 10 tasks/s. Each training episode simulates 100 task arrivals with a new mobility realization drawn from the same distribution.

Test.

After training, agents are evaluated *zero-shot* across 601 test scenarios covering five density regimes: $\{10, 15, 20, 25, 30\}$ tasks/s. No further learning or adaptation occurs during testing.

Baselines.

We compare against: *Always-Local* (never offloads), *Greedy* (always offloads to the V2V peer with the shortest estimated JCT, an optimistic oracle baseline), *Greedy-Fair* (greedy with Jain fairness constraint), *Random* (offloads to a randomly selected peer), *SAC-Base* (SAC with a raw 11-dimensional state vector, no TOPSIS), *PPO-TOPSIS* (on-policy PPO with the same 6D TOPSIS state), and *TD3-TOPSIS* (off-policy TD3 with the same 6D TOPSIS state).

Metrics.

- **Deadline compliance rate**: fraction of tasks with $\mathrm{JCT}_j \le D_j$.
- **Success-Weighted Quality (SWQ)**: $\mathrm{SWQ} = \hat{\tau} \cdot \overline{\left[(D_j - \mathrm{JCT}_j)/D_j\right]}_{\mathrm{success}}$, which rewards meeting deadlines with large slack.
- **Mean JCT** and **total energy consumed**.

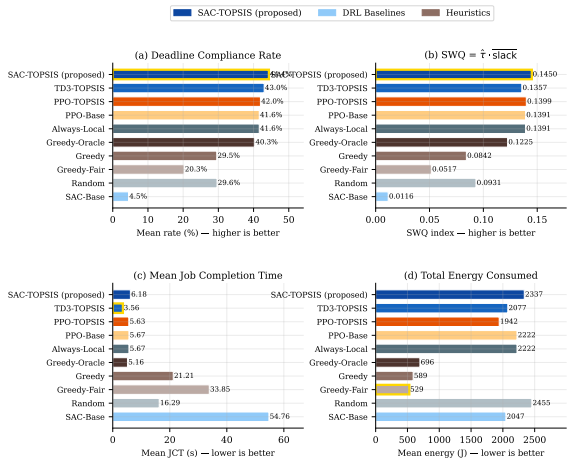

Fig. 2 Performance comparison across key metrics. SAC-TOPSIS (dark blue) achieves the highest deadline compliance rate and SWQ index. The gold border marks the best value in each chart.

Table 1 Aggregate performance over 601 test scenarios (mean over scenarios, densities $\{10, 15, 20, 25, 30\}$ tasks/s). Arrows (↑) show the gain from adding TOPSIS to each algorithm. Bold denotes the best DRL value; † marks the oracle baseline.

Policy	Deadline (%)	SWQ	JCT (s)	Energy (J)
SAC-Base	4.47	0.0116	54.76	2047
SAC-TOPSIS (ours) ↑ +**39.9**	**44.36**	**0.1450**	**3.53**	2222
TD3-Base	41.62	0.1391	5.67	2222
TD3-TOPSIS ↑ +1.4	43.01	0.1357	5.63	**1942**
PPO-Base	41.61	0.1391	5.67	2222
PPO-TOPSIS ↑ +0.3	41.95	0.1399	5.67	2222
Always-Local	41.61	0.1391	5.10	696
Greedy-Oracle†	40.31	0.1225	2.21	589
Greedy	29.54	0.0842	33.85	529
Greedy-Fair	20.26	0.0517	16.29	2455
Random	29.61	0.0931	54.76	2047

5.2 Overall Performance Comparison

Figure 2 and Table 1 summarize the aggregate performance over all 601 test scenarios.

TOPSIS consistently improves every DRL family.

Table 1 reveals a result that is central to this paper: pairing TOPSIS with any tested DRL algorithm improves deadline compliance over the corresponding base version. The magnitude of the gain, however, varies substantially across algorithm families and reflects the interaction between the TOPSIS state and each algorithm's exploration strategy.

SAC experiences the most dramatic improvement: SAC-Base achieves only 4.47% compliance because it converges to an almost-always-offload policy (94.9% V2V), incurring prohibitive queueing delays whenever neighbors are congested. Adding the TOPSIS state rescues the policy to 44.36% (+**39.9** pp), the best result across all evaluated policies. The 6D representation provides the semantic signal—urgency ratios and queue occupancies—that SAC needs to distinguish beneficial from harmful offload decisions.

TD3 already collapses to always-local in its base form (41.62%, identical to Always-Local), because its deterministic policy avoids the risky V2V path without exploration pressure. With the TOPSIS state, TD3 learns to exploit V2V opportunistically (23.4%), raising compliance to 43.01% (+1.4 pp) and achieving the lowest mean JCT (3.56 s) of any policy.

PPO also collapses to always-local in its base form (41.61%) due to high on-policy gradient variance. The TOPSIS state provides marginal improvement to 41.95% (+0.3 pp). The gain is limited because PPO's on-policy nature prevents it from fully exploiting the richer state: variance in the gradient estimates still pushes the policy toward the safe local action despite the informative s_2 (offload-advantage) signal.

Overall, SAC-TOPSIS achieves the highest deadline compliance (44.36%) and SWQ (0.1450) across all policies, including the oracle Greedy baseline (40.31%). These results confirm that *the TOPSIS state is the primary performance driver*, not the choice of DRL algorithm: all three families benefit, and the residual differences among them are secondary effects of their respective exploration mechanisms.

5.3 Zero-Shot Generalization

Figure 3 shows the deadline compliance rate as a function of vehicular density. SAC-TOPSIS consistently leads across all densities, maintaining $> 34\%$ compliance even at the most congested unseen regime (30 tasks/s), where contention degrades the Greedy heuristic to only

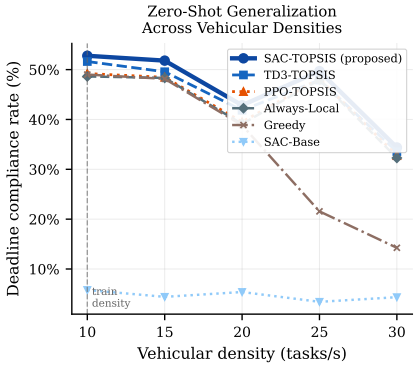

Fig. 3 Deadline compliance rate as a function of vehicular density. All agents were trained only at 10 tasks/s (dashed vertical line). SAC-TOPSIS (solid circles) consistently outperforms all baselines without density-specific retraining.

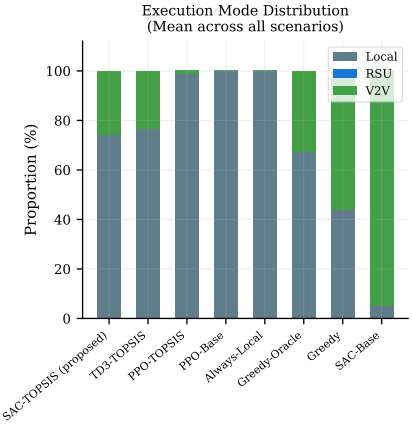

Fig. 4 Distribution of execution modes. SAC-TOPSIS adaptively balances local and V2V execution, while SAC-Base over-offloads to the RSU.

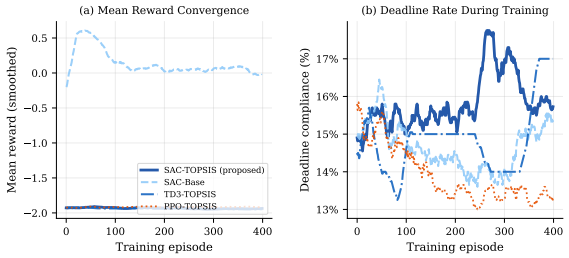

Fig. 5 Training convergence over 400 episodes. (a) Mean reward (smoothed with a 20-episode window); (b) deadline compliance rate during training. SAC-TOPSIS achieves stable convergence in both metrics.

14%. The performance gap over Always-Local ranges from +1.73 pp at 25 tasks/s to +4.15 pp at 10 tasks/s, confirming that the benefit of learning an adaptive policy is most pronounced at lower congestion, where opportunistic V2V offloading is more reliably available.

The scale-invariance of the 6D state vector is the key enabler: because s_3–s_4 are *ratios relative to the deadline*, and s_5–s_6 are *normalized queue occupancies*, the agent's learned policy transfers directly to environments with more or fewer candidates. SAC-Base, by contrast, encodes absolute queue lengths and raw frequency values that lose their meaning under distribution shift.

5.4 Execution Mode Distribution

Figure 4 shows the mean proportion of local, RSU, and V2V executions per policy. SAC-TOPSIS exploits V2V offloading (25.9%), which provides lower latency than the RSU when a peer vehicle is close. Crucially, the TOPSIS module ensures that V2V is chosen only when it offers a better TOPSIS score than the RSU, preventing load migration to overloaded peers.

5.5 Training Convergence

Figure 5 compares the training curves of SAC-TOPSIS, SAC-Base, TD3-TOPSIS, and PPO-TOPSIS. All agents are trained for 400 episodes in the 10 tasks/s scenario. SAC-TOPSIS and TD3-TOPSIS exhibit faster reward improvement and less variance than SAC-Base, confirming that the TOPSIS state provides a more informative learning signal. PPO-TOPSIS converges to a local optimum with lower mean reward, consistent with its collapsed always-local policy at test time.

5.6 Ablation Study: TOPSIS as a Universal State-Engineering Primitive

Figure 6 presents a cross-family ablation in which each DRL algorithm is evaluated both with its raw base state and with the 6D TOPSIS state, keeping all other hyperparameters identical. The arrows annotating each pair quantify the gain attributable solely to the state representation change.

Three findings stand out. First, the gain is *monotone*: TOPSIS never hurts any algorithm.

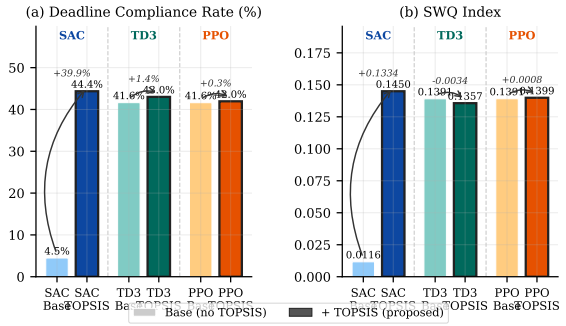

Fig. 6 Cross-family ablation: Base (no TOPSIS, lighter bars) vs. +TOPSIS (darker bars with black border) for SAC, TD3, and PPO. Curved arrows show the gain from adding TOPSIS to each family. The gain is consistent across all families, confirming that TOPSIS state compression is a universal performance driver.

Second, the gain is *largest for the most unstable base policy* (SAC-Base collapses, SAC-TOPSIS recovers), which confirms that the TOPSIS state solves the representation collapse problem rather than merely fine-tuning an already-reasonable policy. Third, the gain is *algorithm-agnostic*: even TD3, which has a well-behaved (if conservative) base policy, improves by +1.4 pp after the state change. This last point is key: it shows that the TOPSIS state carries useful information beyond what any base algorithm can infer from raw observations, and that this information is exploitable by different learning paradigms.

5.7 SAC-Internal Ablation: Causal Decomposition

To understand *which* design choices within SAC-TOPSIS contribute most to its performance, we conduct a fine-grained ablation in which each variant adds exactly one component relative to SAC-Base. Table 2 and Figure 7 summarize the results.

Three complementary insights emerge from this decomposition.

(1) Binary action is the single largest lever (+34.1 pp, SAC-A1). Reducing the action space from three classes (LOCAL/R-SU/V2V) to a binary decision (LOCAL vs. rank-1 offload) largely resolves the collapse of SAC-Base. Without TOPSIS, the agent still must evaluate

Table 2 SAC-internal ablation. Each row adds one design choice over SAC gain in deadline rate relative to SAC-Base.

Variant	Change vs. SAC-Base	Deadline
SAC-Base	— (baseline)	
SAC-A1	Binary action only	
SAC-R1	Deadline-aware reward only	
SAC-S1	Compact 6D manual state only	
SAC-T1	TOPSIS 6D state, 3-action	
SAC-TOPSIS	Full (TOPSIS 6D + binary + deadline)	

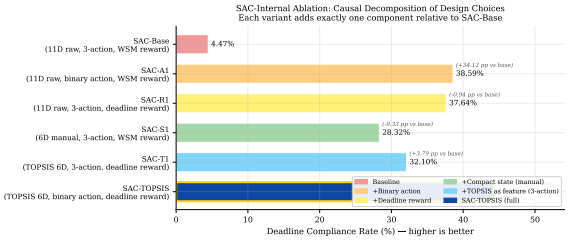

Fig. 7 SAC-internal causal decomposition. Each bar corresponds to a variant that isolates one design choice. All gains are reported relative to SAC-Base (4.47%). The full SAC-TOPSIS combines all three components and achieves the highest compliance rate.

candidates based on raw features, but the simplification of the decision boundary already prevents the always-V2V degenerate policy.

(2) Deadline-aware reward shaping is equally critical (+33.2 pp, SAC-R1). Replacing the Weighted Sum Method (WSM) reward with the continuous deadline-proportional reward of Equation (9) provides a gradient signal even in the failure regime, enabling the agent to learn to minimize tardiness rather than merely avoid it. This gain is independent of the state representation, demonstrating that reward engineering is a standalone contribution.

(3) TOPSIS state provides compounding gains on top of either alone. SAC-S1 shows that compacting the state to 6D manually (without TOPSIS) already yields +23.8 pp over SAC-Base, confirming that feature engineering matters. SAC-T1 shows that using TOPSIS to construct the 6D state (instead of ad-hoc features) raises this to +27.6 pp, even when the action space is still 3-class. The full SAC-TOPSIS, which combines the TOPSIS state with the binary action and the deadline reward, achieves +39.9 pp—more

than any individual component alone—confirming that the three design choices are *synergistic* rather than merely additive.

6 Conclusion and Future Work

This paper presented **SAC-TOPSIS**, a hybrid architecture for deadline-aware task offloading in VEC networks. The key innovation is to use TOPSIS not as a standalone decision maker but as a *state-engineering module* that converts a variable-size, multi-criteria candidate set into a fixed 6D scale-invariant representation. A maximum-entropy SAC agent trained on this representation learns a binary local/offload policy that: (i) achieves a deadline compliance rate of 44.36%, the highest among all evaluated DRL and heuristic policies across 601 test scenarios; (ii) attains a SWQ index of 0.1450; (iii) generalizes zero-shot to unseen vehicular densities (10–30 tasks/s) without retraining; and (iv) avoids the mode-collapse failures of PPO-TOPSIS (always-local) and SAC-Base (always-V2V offload).

A cross-family ablation study confirms that the TOPSIS state compression is the primary performance driver: it improves deadline compliance for *all* tested DRL families—SAC (+39.9 pp), TD3 (+1.4 pp), and PPO (+0.3 pp)—independently of their base learning algorithm. The residual advantage of SAC over TD3 (+1.35 pp) is a secondary effect of entropy-regularized exploration, which prevents the policy from collapsing into the conservative always-local strategy that afflicts deterministic off-policy methods at higher densities.

Future work.

We plan to extend the TOPSIS module to *Fuzzy-TOPSIS*, incorporating linguistic uncertainty into the criterion weights to handle delayed telemetry and noisy channel estimates. We will also investigate multi-agent extensions in which each vehicle in $\mathcal{V}(t)$ runs its own SAC-TOPSIS agent with decentralized coordination. Finally, hardware-in-the-loop evaluation with real 5G sidelink radios is planned to validate the energy model under physical channel conditions.

References

[1] Uddin, A., Zhang, N., Sakr, A.H.: Intelligent offloading in vehicular edge computing: A comprehensive review of deep reinforcement learning approaches and architectures. arXiv preprint arXiv:2502.06963 (2025). Comprehensive Survey on DRL-based offloading for VEC: single-agent, multi-agent, latency, energy, fairness

[2] 3GPP: Release 17 overview. Technical Report Technical Specification Group Services and System Aspects, 3rd Generation Partnership Project (3GPP) (2022). Accessed: 2025-01-01

[3] Zhou, Z., Chen, X., Li, E., Zeng, L., Luo, K., Zhang, J.: Edge intelligence: Paving the last mile of artificial intelligence with edge computing. Proceedings of the IEEE **107**(8), 1738–1762 (2019) https://doi.org/10.1109/JPROC.2019.2918951

[4] Zhang, K., Mao, Y., Leng, S., He, L., Zhang, Y., Peng, X., Vinel, A.: Energy-efficient offloading for mobile edge computing in 5g networks. IEEE Wireless Communications **25**(2), 20–27 (2018) https://doi.org/10.1109/MWC.2018.8370212

[5] Mao, Y., You, C., Zhang, J., Huang, K., Letaief, K.B.: A survey on mobile edge computing: The communication perspective. IEEE Communications Surveys & Tutorials **19**(4), 2322–2358 (2017) https://doi.org/10.1109/COMST.2017.2745201

[6] Fang, J., Shi, J., Lu, S., Zhang, M.: An efficient computation offloading strategy with mobile edge computing for iot. Micromachines **12**(2), 204 (2021) https://doi.org/10.3390/mi12020204

[7] He, Y., Ren, J., Yu, G., Cai, Y.: D2D Communications Meet Mobile Edge Computing for Enhanced Computation Capacity in Cellular Networks. IEEE Transactions on Wireless Communications **18**(3), 1750–1763 (2019) https://doi.org/10.1109/TWC.2019.2896999

[8] Miriyala, S., Chirra, V.R.: Deep learning approaches for computation offloading in edge computing: A critical review. Telecommunication Systems **89**(42) (2026) https://doi.org/10.1007/s11235-026-01426-y

[9] Ahmed, M., et al.: A survey on vehicular task offloading: Classification, issues and future

challenges. Journal of King Saud University – Computer and Information Sciences (2022). Elsevier / ScienceDirect
[10] Li, Z., Zhu, Q.: Genetic algorithm-based optimization of offloading and resource allocation in mobile-edge computing. Information **11**(2), 83 (2020)
[11] You, Q., Tang, B.: Efficient task offloading using particle swarm optimization algorithm in edge computing for industrial internet of things. Journal of Cloud Computing **10**(1), 41 (2021)
[12] Kishor, A., Chakarbarty, C.: Task offloading in fog computing for using smart ant colony optimization. Wireless personal communications **127**(2), 1683–1704 (2022)
[13] Shi, W., Chen, L., Zhu, X.: Task offloading decision-making algorithm for vehicular edge computing: A deep-reinforcement-learning-based approach. Sensors **23**(17), 7595 (2023)
[14] Sousa Vieira, A.S., Souza, A.B., Celestino Júnior, J.: FOFF: an energy-efficient task offloading in VEC-enabled vehicular networks using fuzzy TOPSIS. Cluster Computing (2025) https://doi.org/10.1007/s10586-025-05027-3
[15] Liu, J., Liu, X.: Energy-efficient allocation for multiple tasks in mobile edge computing. Journal of Cloud Computing **11**, 71 (2022) https://doi.org/10.1186/s13677-022-00342-1
[16] Padakandla, S.: A survey of reinforcement learning algorithms for dynamically varying environments. ACM Comput. Surv. **54**(6) (2021) https://doi.org/10.1145/3459991
[17] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. (2017)
[18] Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: International Conference on Machine Learning, pp. 1861–1870 (2018). PMLR